

collection: "Agentic AI — O Futuro ou o Início da Revolução" volume: I title: "Genesis — O Nascimento dos Agentes" subtitle: "Do Primeiro Sopro ao Primeiro Ato: a Arqueologia, a Faísca e a Definição Operacional de Agentic AI" edition: 1.0.0 issued: 2026-06-07 authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR

Agentic AI — O Futuro ou o Início da Revolução

Volume I — GENESIS

Do Primeiro Sopro ao Primeiro Ato: a Arqueologia, a Faísca e a Definição Operacional de Agentic AI

Primeiro tomo da pentalogia Agentic AI — Coletânea MMN AI-to-AI / Nexus HUB57 (2026).

Sobre este ebook

No princípio era o prompt — e o prompt estava com o usuário, e o prompt era estático. Por anos, a interação com IA foi uma forma sofisticada de oráculo: você pergunta, ele responde, você decide, você age. Em algum momento entre 2022 e 2024, essa relação se quebrou. Os modelos passaram a **agir**. Pegaram ferramentas. Navegaram interfaces. Executaram código. Cometeram erros e se corrigiram. Persistiram em objetivos por minutos, depois por horas, depois por dias. Não eram mais oráculos. Eram **agentes**.

Este volume é a **Gênese** dessa transição. Não a Gênese mítica, mas a operacional — datada, técnica, controversa. Você vai descobrir de onde veio a ideia (mais antiga do que imagina), por que falhou repetidamente, o que finalmente fez ela funcionar (LLMs como núcleo cognitivo + tool use + loops de execução), e como definir Agentic AI de forma que distinga o real do hype. Este é o **livro do começo**. Os quatro volumes que seguem dependem do vocabulário que se forma aqui.

Sumário

- 1. Antes da Faísca — A Pré-História dos Agentes (1955-2020)
- 2. O Primeiro Sopro — AutoGPT, BabyAGI e o Verão de 2023

- 3. A Definição Honesta — O Que É e o Que Não É Agentic AI
- 4. As Quatro Capacidades Fundadoras
- 5. Arquiteturas Canônicas dos Primeiros Agentes
- 6. Os Quatro Fracassos Históricos e o Que Aprendemos
- 7. O Cérebro Cognitivo — Por Que LLMs Funcionaram Onde Symbolic AI Falhou
- 8. As Mãos do Agente — Tool Use Como Salto Civilizacional
- 9. O Loop ReAct, o Loop CodeAct, o Loop Planejado
- 10. A Faísca de 2026 — Por Que Agora É Diferente
- Checklist Genesis · Glossário · Próximo Volume

1. Antes da Faísca — A Pré-História dos Agentes (1955-2020)

A palavra **agente** em IA não nasceu em 2023. Nasceu em 1955, no Dartmouth Summer Research Project on AI, onde McCarthy, Minsky, Rochester e Shannon escreveram a proposta seminal. Mas o conceito moderno de **agente racional** foi codificado por Stuart Russell e Peter Norvig em Artificial Intelligence: A Modern Approach (1995):

"Um agente é qualquer coisa que pode ser vista como percebendo seu ambiente através de sensores e atuando sobre esse ambiente através de atuadores."

Essa definição dominou décadas. Mas o **agente** dos livros-texto era, na prática, uma **abstração filosófica**. Agentes reais — sistemas que percebiam e agiam autonomamente em domínios abertos — não funcionavam. Por quê?

Três paradigmas tentaram, três falharam:

Symbolic AI (1955-1990): agentes baseados em lógica formal, regras explícitas, ontologias. Bonitos no papel. Frágeis no mundo real. O "common sense reasoning" exigia capturar conhecimento de mundo em regras — projeto Cyc consumiu 40 anos e bilhões de regras para resolver problemas que crianças resolvem.

Reinforcement Learning clássico (1990-2015): agentes que aprendiam por recompensa em ambientes específicos. Funcionou esplendidamente em jogos (Atari, Go, StarCraft). Falhou em transferir para mundo aberto. Cada novo ambiente exigia retreinamento massivo.

Multi-Agent Systems acadêmicos (1995-2020): arquiteturas BDI (Belief-Desire-Intention), FIPA protocols, agent communication languages como KQML. Produziu papers e teses. Nunca produziu produto.

A constante das três falhas? Falta de **núcleo cognitivo generalista**. O agente sabia agir; não sabia entender. Em 2020, ninguém na indústria construía produtos com a palavra "agente" no nome. Era termo de pesquisa, não de mercado.

2. O Primeiro Sopro — AutoGPT, BabyAGI e o Verão de 2023

A faísca foi pequena, anônima e improvável. Em março de 2023, **Toran Bruce Richards**, desenvolvedor britânico, publicou no GitHub um projeto chamado **AutoGPT**. A ideia era simples: usar GPT-4 não para responder uma pergunta, mas para **decompor uma meta em subtarefas, executá-las uma a uma, e iterar até a meta ser alcançada**.

Em uma semana, AutoGPT ultrapassou 100 mil estrelas no GitHub. Em um mês, era o repositório mais comentado da história até então. Vídeos no YouTube mostravam o sistema fazendo coisas que nenhum chatbot fazia: pesquisar na web, escrever código, executá-lo, ler o output, corrigir erros, tentar de novo.

Funcionava? Mal. AutoGPT entrava em loops infinitos. Gastava centenas de dólares em chamadas de API tentando resolver tarefas triviais. Travava em tarefas complexas. Mas demonstrou algo inegável: **o LLM podia ser o cérebro de um agente real**, e o resto era engenharia.

Paralelamente, **Yohei Nakajima** publicou o **BabyAGI** — versão minimalista, 140 linhas de código Python, com uma fila de tarefas e três funções principais (execução,

priorização, criação). BabyAGI virou template educacional global. Toda startup de IA agêntica em 2023-2024 começou com um fork mental dele.

Setembro de 2023: **MetaGPT**, da equipe chinesa Sirui Hong et al., introduziu a ideia de **múltiplos agentes especializados colaborando** — CEO, CTO, engenheiro, QA, todos LLMs com roles diferentes. Mostrou que multi-agente podia funcionar onde mono-agente travava.

Outubro de 2023: **Open Interpreter** (Killian Lucas) levou agentes ao desktop — Claude/GPT executando código localmente, com acesso a arquivos, terminal, browser.

O verão de 2023 foi o **Cambriano dos agentes**. Centenas de projetos, milhares de devs, bilhões em VC. A maioria morreu. Mas as ideias sobreviveram.

3. A Definição Honesta — O Que É e o Que Não É Agentic AI

Antes do hype invadir os decks, precisamos de uma definição operacional. Aqui está a versão honesta usada por engenheiros sérios em 2026:

Agentic AI é um sistema baseado em modelo de fundação (LLM ou multimodal) que: (a) **percebe** um ambiente (texto, imagem, código, mundo digital), (b) **decide** ações em direção a um objetivo declarado, (c) **executa** essas ações através de ferramentas (APIs, código, browsers, robôs), (d) **observa** os resultados e (e) **itera** até completar o objetivo ou exaurir o orçamento.

Quatro propriedades emergem dessa definição:

P1 — Autonomia por etapa: o sistema decide o próximo passo, não o humano.

P2 — Persistência de objetivo: o sistema mantém o objetivo através de múltiplas etapas, mesmo com falhas intermediárias.

P3 — Adaptação a feedback: o sistema modifica plano com base nos resultados observados.

P4 — Uso instrumental: o sistema invoca ferramentas externas como meio, não como fim.

O que NÃO é Agentic AI (apesar do marketing): - **Chatbot multi-turn:** responde, espera, responde, espera. Não age entre turns. Não persegue objetivo próprio. - **RAG simples:** recupera documentos e gera resposta. Não decide passos. Não itera com feedback. - **Workflow LLM hard-coded:** pipeline determinístico com LLM como uma das etapas. A decisão de fluxo é do código, não do modelo. - **Tool use ocasional:** modelo chama uma função e devolve resposta. Sem loop, sem persistência, sem adaptação.

A maioria das aplicações vendidas como "agentic" em 2024-2025 não passavam dessas barras. A maioria em 2026 finalmente passa. Saber distinguir é skill profissional.

4. As Quatro Capacidades Fundadoras

Para que um sistema seja genuinamente agêntico, quatro capacidades precisam coexistir:

C1 — Raciocínio em cadeia (Chain-of-Thought) O modelo deve ser capaz de gerar passos intermediários explícitos. Sem isso, não há plano; sem plano, não há execução estruturada. Marco fundador: paper de **Wei et al. (2022)** mostrando que CoT melhora dramaticamente desempenho em raciocínio matemático.

C2 — Tool use estável O modelo deve invocar APIs externas com schema confiável. JSON bem-formatado, parâmetros corretos, tratamento de erro. Marco: **Function Calling** do OpenAI (junho 2023), depois adotado por Anthropic, Google, e padronizado.

C3 — Memória operacional O agente precisa lembrar do que fez, do que aprendeu, do que tentou. Contexto longo (200K-1M tokens em 2026) cobre a memória de curto prazo. Memória de longo prazo via vector DB + retrieval cobre o resto.

C4 — Auto-avaliação O agente precisa decidir se está progredindo. Quando declarar tarefa concluída. Quando pedir ajuda. Quando desistir. Sem isso, agentes ficam em loop ou param prematuramente.

Modelos pré-2023 não tinham as quatro. Modelos de 2024 começaram a ter as quatro de forma rudimentar. Modelos de 2026 (Claude Opus 4.5, GPT-5, Gemini 2.x Ultra) têm as quatro de forma robusta. **A história agêntica é a história dessas quatro capacidades amadurecendo simultaneamente.**

5. Arquiteturas Canônicas dos Primeiros Agentes

Cinco padrões emergiram entre 2023 e 2026:

A1 — ReAct (Reasoning + Acting) Yao et al., 2022. Loop: Thought → Action → Observation → Thought → O agente alterna entre pensar e agir, com cada ação informada pela última observação. É o padrão mais simples e ainda o mais comum. Bibliotecas como **LangChain**, **LlamaIndex**, **Smolagents** implementam variantes.

A2 — Plan-and-Execute Wang et al., 2023. O agente primeiro gera um plano completo, depois executa cada passo. Funciona melhor para tarefas previsíveis com poucos branches. Pior para domínios voláteis.

A3 — Reflexion Shinn et al., 2023. Após cada tentativa, o agente faz **reflexão verbal** explícita sobre o que deu errado, e essa reflexão entra no contexto da próxima tentativa. Ganhos massivos em tarefas de programação e raciocínio.

A4 — Multi-Agent (CrewAI, AutoGen, MetaGPT) Múltiplos agentes com roles distintos (planner, executor, critic, etc.) colaboram via mensagens. Padrão dominante em produtos sérios em 2026.

A5 — Code-as-Action (CodeAct) Wang et al., 2024. Em vez de chamar tools via JSON, o agente **escreve código Python** que executa as ferramentas como funções. Mais expressivo, mais robusto a tarefas compostas. Padrão emergente em 2025-2026.

Cada padrão tem trade-offs. Não há vencedor universal. Engenheiro agêntico maduro escolhe padrão por tarefa.

6. Os Quatro Fracassos Históricos e o Que Aprendemos

A história agêntica é também a história dos fracassos. Quatro merecem nota:

F1 — A Quimera Symbolic-AI (1980) Sistemas especialistas prometeram automação cognitiva via regras. Falharam porque o mundo tem cauda longa. Lição: **conhecimento implícito não cabe em regras explícitas.**

F2 — A Bolha dos Multi-Agent Systems Acadêmicos (1995-2010) Protocolos elegantes, zero adoção comercial. Lição: **protocolo sem produto morre.**

F3 — A Catástrofe AutoGPT (2023) Loops infinitos, custos absurdos, alucinações cumulativas. Lição: **sem evals e sem guard-rails, agentes são brinquedos caros.**

F4 — A Decepção dos "Devin clones" (2024) Vários startups prometeram "engenheiro IA autônomo". A maioria não entregou. Lição: **demos em vídeo ≠ produto em produção; gap é gigante.**

Cada fracasso pavimentou o avanço. Em 2026, engenheiros sérios sabem: agentes precisam de evals, guard-rails, observabilidade, fallbacks, custos controlados. Isso é o **estado da arte operacional.**

7. O Cérebro Cognitivo — Por Que LLMs Funcionaram Onde Symbolic AI Falhou

Por que LLMs viraram o cérebro agêntico que symbolic AI nunca foi? Resposta em três camadas:

Camada 1 — Conhecimento de mundo embarcado LLMs treinados em corpus massivo internalizaram, implicitamente, milhões de fatos sobre como o mundo funciona. Não precisam que você ensine que "se chove, ruas ficam molhadas" — eles já sabem por correlação. Symbolic AI exigia codificar isso explicitamente.

Camada 2 — Generalização linguística LLMs podem receber instruções em linguagem natural e adaptá-las a casos novos. Isso é qualitativamente diferente de regras fixas. Um agente Claude/GPT consegue interpretar "limpe o desktop separando documentos pessoais dos de trabalho" sem ter sido treinado especificamente para essa tarefa.

Camada 3 — Composição emergente LLMs compõem conhecimentos de domínios diferentes. Um agente pode usar conhecimento médico + conhecimento de programação + conhecimento de UX para construir um app de saúde — sem ter sido programado para fazer essa composição.

Symbolic AI tentou construir composição explícita; LLMs ganharam composição emergente. **Esse é o salto que tornou agentes possíveis.**

8. As Mãos do Agente — Tool Use Como Salto Civilizacional

Se LLM é o cérebro, **tool use** são as mãos. Um cérebro sem mãos é oráculo. Um cérebro com mãos é ator.

Tool use técnica: - **Function calling**: modelo emite JSON estruturado especificando função e argumentos. Sistema executa, devolve resultado. - **MCP (Model Context Protocol)**: padrão Anthropic 2024 para conectar agentes a recursos externos sem código custom. - **Code execution sandboxes**: agente escreve código Python; sandbox executa; output volta. - **Computer use**: Claude/GPT controla mouse/teclado de um VM. Adotado a partir de 2024. - **API ecosystem**: cada SaaS expõe API; agente compõe APIs como músico compõe acordes.

Salto civilizacional: pela primeira vez na história, um sistema cognitivo geral tem acesso programático ao **stack inteiro de tecnologia humana**. Isso muda escala do que IA pode fazer — de respostas para **execuções**.

9. O Loop ReAct, o Loop CodeAct, o Loop Planejado

Três loops dominam 2026:

Loop ReAct (clássico)

```

[User]
[Thought: I should use the calculator tool.]
[Action: calculator(5+3)]
[Observation: 8]
[Thought: The answer is 8.]
[Final Answer: 8]
```

Loop CodeAct

```

[User]
[CodeAct: python: print(5+3)]
[Observation: 8]
```




Loop Planejado (Plan-Execute-Replan)



Cada loop tem variantes (com reflexion, com self-consistency, com voting). A escolha do loop define a personalidade do agente. **Conhecer todos é arsenal básico do engenheiro agêntico.**

10. A Faísca de 2026 — Por Que Agora É Diferente

Por que 2026 é o ano da virada (e não 2023 ou 2024)? Cinco fatores convergiram:

Fator 1 — Modelos confiáveis em raciocínio longo Claude Opus 4.5, GPT-5, Gemini 2.x Ultra sustentam coerência por horas de raciocínio. Modelos pré-2025 quebravam em ~30 min.

Fator 2 — Context windows generosas 200K-2M tokens nativos. Suficiente para manter histórico completo de tarefa multi-hora.

Fator 3 — Tool use industrializado MCP virou padrão. Ecossistema de 1000+ MCP servers disponíveis.

Fator 4 — Custos viabilizados Inference 10x mais barata desde 2023. Prompt caching reduz custo em 90% para system prompts repetidos.

Fator 5 — Frameworks maduros LangGraph, AutoGen, CrewAI, Smolagents, Mastra. Devs não precisam construir loops do zero.

Tudo junto: o que era brinquedo em 2023 é infraestrutura em 2026. **A faísca virou fogo. O fogo está se espalhando. O próximo volume — Exodus — mostra para onde.**

Checklist Genesis · Glossário · Próximo Volume

Você terminou Genesis se consegue: - [] Distinguir Agentic AI de chatbot, RAG e workflow LLM. - [] Nomear as quatro capacidades fundadoras (C1-C4). - [] Citar pelo menos três arquiteturas canônicas (ReAct, Plan-Execute, CodeAct). - [] Explicar por que LLMs funcionaram onde symbolic AI falhou. - [] Listar três fracassos históricos e suas lições. - [] Esboçar um loop ReAct em pseudocódigo.

Glossário Genesis: - **Agentic AI:** sistema com autonomia por etapa, persistência de objetivo, adaptação a feedback, uso instrumental de ferramentas. - **ReAct:** padrão Reasoning + Acting (Yao et al., 2022). - **CodeAct:** padrão onde ações são código Python executado. - **MCP:** Model Context Protocol — padrão de conexão agente↔recurso. - **Tool use:** capacidade do agente de invocar funções externas. - **CoT (Chain-of-Thought):** raciocínio em passos explícitos.

Próximo Volume: EXODUS — A Saída do Laboratório · Como Agentes Foram para Produção em Massa em 2025-2026.

Coletânea Agentic AI · MMN AI-to-AI · Nexus HUB57 · 2026